Algorithm 1: AsyncPPReconfiguration—Live pipeline-parallelism reconfiguration with asynchronous KV cache migration. The algorithm transitions from C_{cur} to C_{tgt} through an *intermediate configuration* C_{int} in which both old and new layers coexist, enabling continuous inference throughout the migration.

Notation:
$C_{\mathrm{cur}}, C_{\mathrm{tgt}}, C_{\mathrm{int}} : r \mapsto \{\ell_1, \dots\}$ maps from rank to set of layer indices (current / target / intermediate PP config)
$\mathcal{M}_{\mathrm{add}}, \mathcal{M}_{\mathrm{del}} : r \mapsto \{\ell_1, \dots\}$ maps from rank to set of layers to add / delete
$\mathcal{M}_{\mathrm{mig}} : r_{src} \mapsto (r_{dst} \mapsto \{\ell_1, \dots\})$ nested map: source rank → destination rank → set of layers to migrate
$B_{\max}(r)$: scalar, max KV blocks rank r can hold;
B_{cur}, B_{new}: scalars, current / post-reconfiguration KV block budget
M_r: scalar, total GPU memory on rank r; $u \in (0,1]$: KV cache utilization ratio
W: scalar, weight memory per layer; P: scalar, KV page (block) size
τ: scalar, convergence threshold (tokens); Δt: scalar, polling interval

Function `MaxBlocks`(r, L)**:**
  return $\left\lfloor \frac{M_r \cdot u - L \cdot W}{L \cdot P} \right\rfloor$

Input: Current PP configuration $C_{\mathrm{cur}} = \{(s_r, e_r)\}_{r=0}^{R-1}$, where each rank r owns layers $[s_r, e_r]$;
target PP configuration $C_{\mathrm{tgt}} = \{(s'_r, e'_r)\}_{r=0}^{R-1}$;
convergence threshold τ (tokens)
Output: true if reconfiguration succeeds; false if infeasible

▷ **Phase 1: Feasibility assessment & intermediate configuration**
$C_{\mathrm{int}} \leftarrow C_{\mathrm{cur}}$; $\mathcal{M}_{\mathrm{add}} \leftarrow \emptyset$
foreach *rank* $r \in [0, R)$ **do**   ▷ build intermediate config C_{int}
  $C_{\mathrm{int}}[r] \leftarrow C_{\mathrm{cur}}[r] \cup C_{\mathrm{tgt}}[r]$ ▷ new layers rank r must add
  $B_{\max}(r) \leftarrow$ `MaxBlocks`$(r,\ |C_{\mathrm{int}}[r]|)$
end
$B_{\mathrm{shrink}} \leftarrow \min_r\ B_{\max}(r)$
$B_{\mathrm{used}} \leftarrow$ current number of KV blocks in use
if $B_{\mathrm{used}} > B_{\mathrm{shrink}}$ **then**
  return false   ▷ insufficient memory for reconfiguration, abort
end

▷ **Phase 2: KV Compaction & Resizing**
; **if** $B_{\mathrm{shrink}} < B_{\mathrm{cur}}$ **then**
  `Collective::CompactKV`(B_{shrink})
  `Collective::ResizeKV`(B_{shrink})
end

▷ **Phase 3: Asynchronous weight loading and KV Cache Migration**(non-blocking, concurrent with inference)
`Collective::AddLayerWeights`$(\mathcal{M}_{\mathrm{add}})$
$\mathcal{M}_{\mathrm{mig}} \leftarrow \emptyset$
foreach $(r_{dst}, A_{r_{dst}}) \in \mathcal{M}_{\mathrm{add}}$ **do**
  foreach *layer* $\ell \in A_{r_{dst}}$ **do**
    $r_{src} \leftarrow$ rank owning layer ℓ in C_{cur}
    $\mathcal{M}_{\mathrm{mig}}[r_{src}][r_{dst}] \leftarrow \mathcal{M}_{\mathrm{mig}}[r_{src}][r_{dst}] \cup \{\ell\}$
  end
end
`Collective::StartKVMigration`$(\mathcal{M}_{\mathrm{mig}})$

▷ **Phase 4: Convergence monitoring**
; **repeat**
  $T_{\mathrm{sched}} \leftarrow$ `getLastScheduledTokenIndex()`
  $T_{\mathrm{applied}}[\,] \leftarrow$ `Collective::GetLastSyncedTokenIndex()`
  allDone $\leftarrow$ true
  foreach *rank* $r \in \mathrm{dom}(\mathcal{M}_{\mathrm{add}})$ **do**
    if $T_{\mathrm{sched}} - T_{\mathrm{applied}}[r] \geq \tau$ **then**
      allDone $\leftarrow$ false
      break
    end
  end
  if *allDone* **then break**
  sleep(Δt)
until

▷ **Phase 5: Commit PP Configuration Change**

foreach *rank* $r \in [0, R)$ **do**   ▷ compute layers to delete from C_{int}
  $\mathcal{M}_{\mathrm{del}}(r) \leftarrow C_{\mathrm{int}}[r] \setminus C_{\mathrm{tgt}}[r]$
  $B_{\mathrm{new}}(r) \leftarrow$ `MaxBlocks`$(r,\ |C_{\mathrm{tgt}}[r]|)$
end
$B_{\mathrm{new}} \leftarrow \min_r\ B_{\mathrm{new}}(r)$
`Sync::SyncAndCommit`$(\mathcal{M}_{\mathrm{del}},\ B_{\mathrm{new}})$ ▷ commit new config, then delete old layers & resize KV
return true